



プログラミング概論I
(オンデマンド授業)



担当: 中山功一 (7号館3階308室)

メールアドレス: knakayama@is.saga-u.ac.jp

2

オンライン授業の注意点

別の動画(以下)を参照
「中山が担当するオンライン授業の注意点」

教育の理念

別の動画(以下)を参照
「ティーチング・ポートフォリオ
～中山の教育の理念～」

3

授業ポータルサイトの
トップページ

科目一覧



理工概論：情報
分野／データサイ
エンス分野



プログラミング
概論1



技術文書作成



授業の注意／中
山研究室紹介



知能情報工学入
門：前半

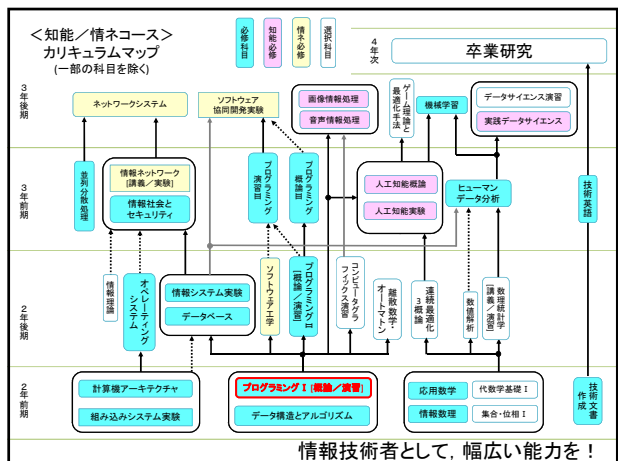


プログラミング
概論2



Advanced
Mathematical and
Information Science
(数理・情報サイエ
ンス概論)

<https://sites.google.com/view/portal-site-nakayama/>



月曜4-5校時: 担当者

プログラミング概論1	中山 功一 knakayama@is.saga-u.ac.jp	7号館 308室
月曜5校時 ／時間外	TAへの質問 ta-gairon@fu.is.saga-u.ac.jp	
プログラミング演習1	福田 修 fukudao@is.saga-u.ac.jp	7号館 206室
月曜4校時 ／集中講義	TAへの質問 ta-enshu@fu.is.saga-u.ac.jp	

試験(対面)の配点予定: 2026

< 配点 > (COM室+AV講義室)

- ・第1中間テスト: 100点+α (5/25(月)4-5校時)
※5/18(月)練習テスト(挽回点20点あり)
- ・第2中間テスト: 100点+α (7/6(月)4-5校時)
- ・期末テスト : 100点+α (8/3(月)4-5校時)
(**演習1**の内容を含む)

成績(素点) = テスト上位2回 ÷ 2

⇒ 概論が不可なら演習も不可(逆も真)

※ 情報・DS分野以外の学生は1科目のみ履修可

※ 時間外クラスの学生は試験は1回のみでOK

< 成績非開示の依頼方法 >

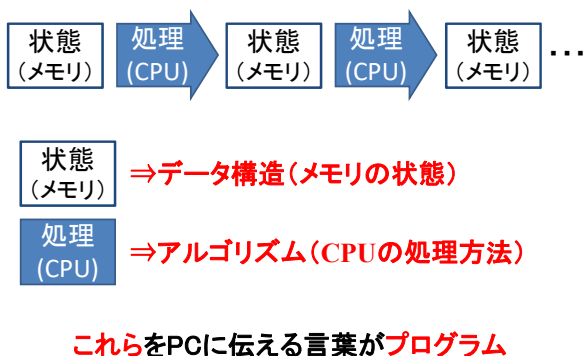
- ・ 個人開示: 教務課(前期9月1日, 後期3月1日)
 - ・ 一斉開示: 教員(中山)から(適宜)
- 一斉開示を希望しない場合, 成績開示日まで
“合否を含めて”成績は開示されません.
“合格まで何点必要か”などの問合せも回答しません.
一斉開示を希望しない人は以下を送信すること.

学籍番号 :
氏名 :
非開示科目 :
私は, 成績開示日(前期:9月1日, 後期:3月1日)まで,
上記科目の成績(途中経過を含む)の開示を希望しません.
また, 成績開示日まで, 自分の成績の開示を依頼しません.

この授業で習うこと

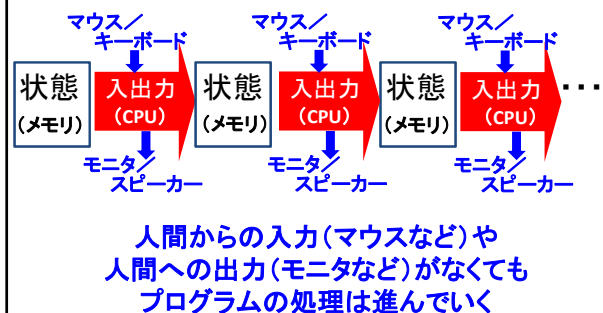
C++言語でプログラミングができるようになる

プログラムとは



プログラムと動作イメージ

状態を変更する方法(=アルゴリズム)を
コンピュータに指示する言葉が**プログラム**



書く！

```
void main (void){
    int i, a=2;
    for(i=0; i<=a; i++){
        ソース
        プログラム
    }
}
```

コンパイルする物
(別のプログラム)を
コンパイラとよぶ

コンパイル

実行形式 プログラム

C++言語の 実行ファイル

注: Pythonは
インタプリタ
(コンパイルしない)

動く！



13

- ・ ソースコード

- C++言語で記述された文字列(実行不可)
 - Visual studio の編集対象
- 実行ファイル**
- 機械語で表現されたビット列(実行可能)
- コンパイル**(compile)
- ソースを実行ファイルに変換する処理
- コンパイラ**(compiler)
- コンパイルを実行する物(プログラム)
- アルゴリズム**
- 問題を解く手順を日本語で表現したもの

14

- ・ プログラム⇒関数の集まり
 - メイン関数およびその他の関数(サブルーチン)
 - 最小のメイン関数 `void main() { }`
- ・ 関数⇒文の集まり 文の例 `cout << a+b;`
- ・ 文⇒トークンの集まり
 - `トークン(空白)トークン(空白)トークン...`
 - 空白や改行は何個でも同じ(二つ目以降は無視)
⇒だから、見やすいように空白や改行を入れる

15

C++では人間が見やすくするため
(コンパイラは無視する)

- 構造を明示するための **段付け**
 - 同一レベル⇒行頭を揃える
 - 下位レベル⇒行頭を下げる
(全体-構成要素の関係)
- 意味の区切りに空行を空ける
 - 異なるアルゴリズムステップの間
 - 変数定義とアルゴリズム記述の間
 - 異なる関数定義の間

プログラマが間違えても
コンパイラは警告しない

16

人間が見やすくするため
(コンパイラは無視する)

- ・プログラムを分かりやすくする注釈
 - デバッグ (誤りの発見・修正) が容易になる
 - プログラムの理解が容易になる(他人も自分も)

注:コンパイラはコメントを無視する

```
// この行はコメント
    次の行はコメントではない
/*   ここから
        ここまでがコメント */
```

17

```
#include <iostream>
#include <ctype.h>
using namespace std;
```

```
void main(void){
    int ch;

    ch = cin.get();

    while(ch != EOF){

        if(islower(ch)) cout << toupper(ch);

        else            cout << ch;

        ch = cin.get();
    }
}
```

コメント不使用

18

```
#include <iostream>
#include <ctype.h>
using namespace std;

void main(void){
    int ch;           // 入力文字

    // 文字を入力する
    ch = cin.get();

    // 入力終了でない限り、以下の処理を繰り返す
    while(ch != EOF){
        // 当該文字が英小文字なら、大文字に変換し出力
        if(islower(ch)) cout << toupper(ch);

        // そうでなければ、当該文字を出力
        else cout << ch;

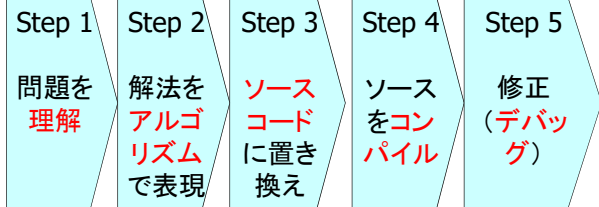
        // 次の文字を入力
        ch = cin.get();
    }
}
```

コメント使用

体系的なプログラミング手法

プログ概論: 基本知識

プログ演習: 実習



アルゴリズム: 解決手順を日本語で表現したもの
コンパイル: ソースを実行ファイルに変換すること

20

プログラムを作る能力
(≡稼ぐ能力)の基礎を
身につけることを目標とする

21

変数・代入

箱の作り方: 変数宣言

箱を作るおまじない

自分で決める

箱の形	箱の名前:
例 int	hako;

箱の形(型)は
C++言語では
自分で選ぶ
(Pythonでは自動)

23

変数名のルール(関数名も同じ)

- 使用可能な文字 (注: 原則, 半角のみ)
アルファベット / 数字 / _ (アンダーバー)
- 禁則
 - 1文字目の数字は不可
 - 予約語も不可
(if/for/whileなど既定のトークン)

24

箱の作り方: 変数宣言

<一つの箱を作成>

変数のデータ型(箱の形)「int」を左に,
変数の名前「answer」を右に書く

```
int answer;
```

<多数の箱を同時に作成>

変数(配列)の型(int)と名前(value)と
箱の数([10])を順番に書く

```
int value[10];
```

25

代入: 値を代入する

- 代入される変数「y」を左に書く
- 代入する値「3」を右に書く
- 両方を「=」でつなぐ

```
y = 3;
```

注意! 「=」はイコールではなく「代入」
比較の「イコール」は「==」と書きます

26

代入: 値を代入する

- 代入するのは,
値「3」でも変数「x」でも良い
(変数の値が代入される)

```
y = 3;
```

```
x = 3;    y = x;
```

- 新たに代入したら上書きされる

```
y = 4;    y = 5;
```

27

宣言と同時に代入する

- 変数の型 変数名 = 代入する値

```
int y = 3;
```

```
int value[10] = {4,5,3,0,1,8,9,3,3,2};
```

(value[0] ~ value[9])

要素番号に負の数は不可
(後ろから数えない)

28

変数操作のまとめ

注: 他の変数や命令語,
予約語と同じ変数名は不可

int a; int型の変数aを定義する
aの初期値は未定義
(ゴミが入ってる)

a = 1; 既に定義されている
変数aに値を代入する

int a = 1; int型の変数aを定義する
aの初期値も同時に代入する

29

代入と参照

- **代入** (例: a = 3; //変数aに3を代入)
 - 変数の値を**変更する**操作
 - 代入すると「上書き」される
(それ以前の値は失われる)
- **参照** (例: cout << a; //変数aの値を表示)
 - 変数の値を**調べる**操作
 - 参照しても、変数の値は変わらない

30

ライブラリ と 名前空間

ライブラリの読み込み

```
#include <iostream>
```

“iostream”(アイオーストリーム:標準入出力)
というファイル(ライブラリ)を
“include”(インクルード:読み込み)します,
という宣言

- ・標準入力 `cin` → キーボードからの入力
- ・標準出力 `cout` → モニタへの表示

※ `#include<stdio.h>`の例 入力 `scanf` / 出力 `printf`

名前空間(ネームスペース)

```
using namespace std; //std::を省略
```

小さなプログラムでは1つの名前空間
→標準名前空間「std::」を全体で使用

プログラミング概論・演習 1/2では不要

大きなプログラムでは複数の名前空間
→Aチームは名前空間「library::」を使用
→Bチームは名前空間「livecampus::」を使用
→Cチームは名前空間「kyoumu::」を使用, など

標準的な何もしないプログラムの例

```
#include <iostream> //標準入出力
using namespace std; //標準名前空間
```

```
int main(){
    //何もしない
    return 0; //正常終了
}
```

変数の型

箱の作り方: 変数宣言

箱を作るおまじない

自分で決める

箱の形	箱の名前:	
例 int	hako;	整数: {1, 3, -7}
double	value;	実数: {2.3, 5.88}
char	answer;	文字: {a, Y, ?}
bool	box2;	論理: {0, 1}

C++言語では
自分で選ぶ

データ型 (数値: 整数型) 注: 実数型は来週

- 変数が格納するデータの種類を表す
 - `int`: 整数型 (4バイト: $-2^{31} \sim 2^{31}-1$)
 - 授業で最も良く使う
 - `unsigned int`: 正の整数型 (4バイト: $0 \sim 2^{32}-1$)
 - `short`: 整数型 (2バイト: メモリ節約用)
 - `long`: 整数型 (4バイト)

注: 整数型は小数点以下“切り捨て”
例: (int) $5 \div 2 = 2$ (2.5ではない)

37

数値表現

- 10進数 (そのまま)
 - 1, 2, 4, 8, 16, 32, 64, ...
- 8進数 (頭に0, あまり使わない)
 - 01, 02, 04, 010, 020, 0100, ...
- 16進数 (頭に0x, または0X)
 - 0x1, 0x2, 0x4, 0x8, 0x10, 0x20, ...

本授業では(原則)10進数を利用

38

多次元配列

配列

- 複数の変数を、まとめて扱う仕組み
- イメージ (1000個の変数):
 - 普通の変数で定義:
`name1, name2, ..., name1000`
 - 配列で定義:
`name[0], name[1], ..., name[999]`
- 配列なら,

```
for(int i=0; i<N; i++)  
    cout << name[i] << endl;
```

のように書ける (添え字部分に変数を使える)

変数が
1000個

変数は1つ
添え字で区別

40

一次元配列の定義

配列名
要素数
要素のデータ型

配列定義
の3要素

※ 要素数はコン
パイル時に決定
(途中で変更不可)

[文法]

要素のデータ型 配列名[要素数];

[例]

```
int scoreList[100]; // 点数一覧  
double temperatureList[31*24*60*60];  
// 1ヶ月の気温 (1秒毎)
```

41

一次元配列の初期化

プログラムの実行開始時に初期値を設定
要素の型 配列名[要素数] = {初期値リスト};

定義と場合と同じ

[例]

```
int table[5] = {1, 5, 32, 7, 3};  
double list[] = {3.2, 5.4, 2.65, 4.0};
```

※ 要素数をコンパイラが計算してくれる

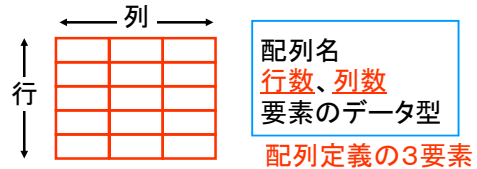
配列の各要素のアクセス

- 要素数Nの一次元配列
 - 最初の要素 : 要素番号 0
 - 2番目の要素: 要素番号 1
 - :
 - N番目の要素: 要素番号 N-1
 - 配列の各要素の参照方法
配列名[要素番号]
- <例> Answer = table[3];

43

二次元配列

- (複数の) 値を格納する箱
- 1つの配列は複数の要素を持つ
- 要素のデータ型は同一
- 各要素が1つの値を格納(変数と同等)



44

二次元配列の定義

配列定義の3要素

配列名
行数、列数
要素のデータ型

[文法]

要素のデータ型 配列名[行数][列数];

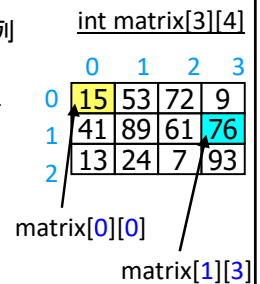
[例]

int matrix[5][4]; // 5行4列の配列

45

配列要素のアクセス

- 行数N、列数Mの二次元配列
 - 行番号の範囲: 0, ..., N-1
 - 列番号の範囲: 0, ..., M-1
- 配列の各要素は
配列名[行番号][列番号]
でアクセス



<例>
Answer = matrix[2][3];

46

二次元配列の初期化

- 二次元配列を定義すると同時に値を代入

[例]

```
int matrix[3][4] = {
    {15, 53, 72, 9},
    {41, 89, 61, 76},
    {13, 24, 7, 93}
};
```

15	53	72	9
41	89	61	76
13	24	7	93

47

N次元配列

- 図では書けませんが, , ,

[文法]

データ型 配列名[要素数][要素数]...[要素数];

[例]

```
int table [31] [24] [60] [60];
// 何日何時何分何秒のデータ
```

データ量は以下とほぼ同じ

```
int table [31 * 24 * 60 * 60];
```

48

配列を用いたプログラム例

- 各要素の参照・代入は変数と同様

```
table[0] = 1;
table[i] = table[i-1]*2;
```

- 配列要素は変数を使ってアクセス

```
double array[10];
for (int i=0; i<10; i++) {
    cout << array[i];
}
```

49

配列を用いたプログラム例

- 全て1に初期化

```
int data[10000];
for(int i=0; i<10000; i++) data[i]=1;
```

- N次元ベクトルの内積を計算

```
double vect_a[10] = { .... }; // ベクトルa
double vect_b[10] = { .... }; // ベクトルb
double inner_product = 0; // aとbとの内積
for(int i=0; i<10; i++)
    inner_product += vect_a[i] * vect_b[i];
```

50

入出力

入力: キーボードから入力する

- 入出力ライブラリの指定
(プログラムの先頭で: 1回だけでOK)

```
#include <iostream>
using namespace std;
```

- キーボードから値を入力し
変数に代入する

```
cin >> 変数;
```

52

出力: 画面に表示する1

- 入出力ライブラリの指定(先頭で)

```
#include <iostream>
using namespace std;
```

- 値を表示する

```
cout << 321; //表示したい値
```

- 1文字を表示する

```
cout << 'a'; //表示したい1文字
```

- 文字列を表示する

```
cout << "こんにちは"; //表示したい文字列
```

321

a

こんにちは

出力: 画面に表示する2

- 変数の値を表示する

```
int a = 5; cout << a;
```

- 続けて表示する

```
cout << 321 << 'a' << "bcd";
```

- 改行する

```
cout << 'a';
cout << endl;
cout << "bcd";
cout << endl << endl;
cout << 'c';
```

5

321abcd

a
bcd
c

54

バグとコンパイルエラー

コンパイルエラーについて

- **コンパイラのエラーメッセージとは？**
 - 誤りの原因(バグ)を特定するヒント
(場所:何行目か? / 内容:どんなバグか?)
 - 先頭のものから順番に対処すること
 - 注:コンパイルが通ってもエラーの可能性あり

※ ステップ実行(1ステップずつ実行)でバグ取り

- **F10** キー(ステップオーバー)を押す
- 矢印が示す行は、**次に実行する行**
- 表示値は当該ステップ**実行直後**の値

56

コンパイルエラーについて

- **プログラムは一字でも間違えるとエラー**
 - 半角と全角は違う文字
“a”, “a” **全角無用**
 - 大文字と小文字は違う文字
“a”, “A” **原則小文字**
 - 全角空白も見えない文字
“”, “ ” **全角空白禁止**
全角スペースを表示しない
エディタ(Visual Studio)に注意
(怪しければ全角スペースで検索)

57

良くあるバグの原因

- 文字の打ち間違い
 - **O**と**0**(大文字と数字),
 - **I**と**l**と**1**と**!**(大文字,小文字,数字,記号)
 - **,**と**.**(カンマとピリオド)
 - **;**と**:**(セミコロンとコロン)
- 括弧の非対応
 - **(**と**)**、**{**と**}**、**[**と**]**、**"**と**"**、**'**と**'**
- **;**の打ち忘れ

C++言語では
特に注意

58

デバッグ／デバグ

デバグの基本

- プログラムが正しく動作しない場合
 - **デバグ**を立ち上げる
 - **バグ**(意図通りでない箇所)を発見する
 - **デバグ**(バグを修正)する
- **デバグ**:デバッグのためのソフトウェア
 - 実行中の変数値を表示(トレース)する

60

デバッガの基本

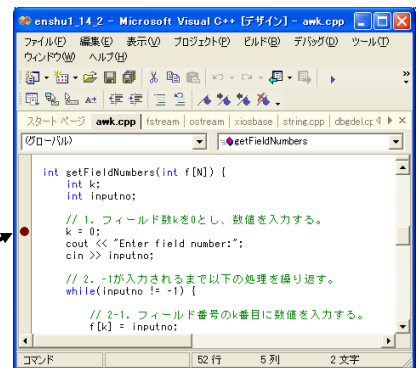
- 機能1: ブレークポイント
 - 確認したい箇所まですばやく実行できる
 - プログラム中に複数個指定できる

61

ブレークポイント(その1)

指定行で
無条件に
一時停止

指定行

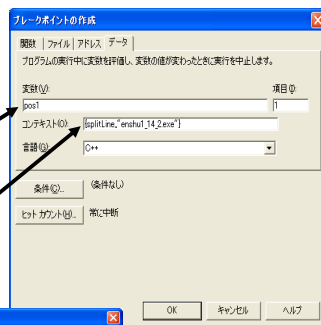


ブレークポイント(その2)

指定した変数が
変更された時に
一時停止

指定変数

定義されている関数
実行形式プログラム



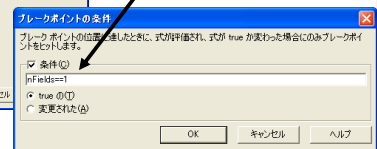
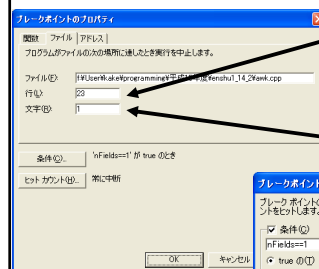
63

ブレークポイント(その3)

指定行で指定した条件時に一時停止

指定行

指定条件



デバッガの基本

- 機能1: ブレークポイント
 - 確認したい箇所まですばやく実行できる
 - プログラム中に複数個指定できる
- 機能2: ステップ実行
 - プログラムを1文実行する毎に停止する
 - 文の実行順序が正しいか確認する
 - 短所: 時間がかかる

ステップオーバー
(F10)

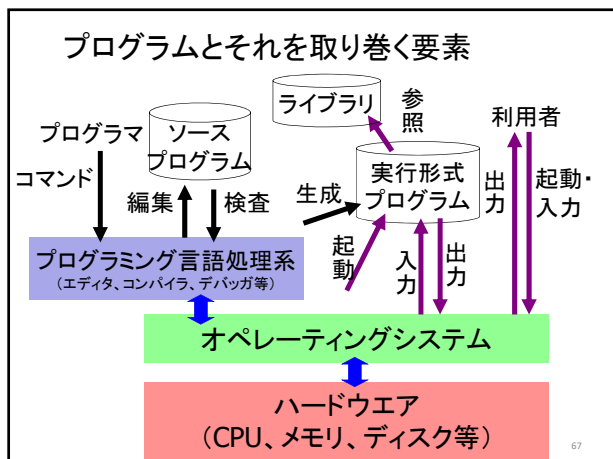
65

デバッグの方法

- 1行ずつ追いかけるときはステップ実行
- アタリをつけるときはブレークポイント

状況に応じて使い分ける

66



中間テストまでに習う基本機能

① 利用者とのやり取り

- 変数への入力
- 値の出力

② データの処理

- 式を用いた値の計算
(四則演算、比較等)
- 変数への値の代入

③ データの定義

- 変数
- 定数

④ 処理順序の制御

- 条件分岐: **if文**, **switch文**
- 繰り返し: **while文** (繰り返し回数が未知の場合)
- 繰り返し: **for文** (繰り返し回数が既知の場合)

⑤ 定型処理をまとめる

- ## 一 関数

クロージング テスト

今回、習ったことを復習するために、
クロージングテストを行います

問題ごとに動画を止めて、答えをメモしましょう。

最後に、答えを動画で表示しますので、
自分が理解しているか、確認しましょう。

クロージング テスト

次のプログラムについて答えなさい

問1 : 入力された数字が入る変数は？

問2～4:画面に表示される文字は？

```
#include <iostream>
using namespace std;
int a = 1, b = 2, c = 3;
cin >> a;
cout << b; //問2
cout << 'c'; //問3
cout << "Hello"; //問4
cout << endl;
```

＜選択肢＞

- ① 1
② 2
③ 3
④ a
⑤ b
⑥ c
⑦ その他

解答

- 【1】 ④ aに入力される
- 【2】 ② bは、初期値から変換なし
- 【3】 ⑥ ‘’で囲われているので、
値:3ではなく文字:cが表示される
- 【4】 ⑦ 文字列:Helloが表示される

クロージング テスト

次のプログラムについて答えなさい

問5 : 最終的な変数aの値は？

問6 : 最終的な変数bの値は？

問7 : 最終的な変数cの値は？

```
int a = 1, b = 3, c;  
c = a;  
a = b;  
b = 5;
```

＜選択肢＞

- ① 1
② 2
③ 3
④ 4
⑤ 5
⑥ 0
⑦ その他

解答

【5】③ bの値:3が代入されている

【6】⑤ 最後に5が代入されている

【7】① aの値:1が代入されている

お疲れさまでした

